## ■ 1.9.6 Special Topic: External Programs

*Mathematica* does many things well. There are, however, some things that are still best done outside of *Mathematica*. In such cases, you can call external programs from within *Mathematica*, and then, for example, get the results back into *Mathematica* for further processing.

One situation in which it is often convenient to use external programs is in working with special input or output devices. You can use an external program to control the device, and then transfer arrays of *Mathematica* data to or from the device.

If you already have an external program that does a particular calculation, it may be easier for you to do that calculation with your existing program, rather than to recode the algorithm in *Mathematica*. Even if the main part of your calculation is done by an external program, you can still use *Mathematica* to prepare the input, and to process the output.

On most computer systems (such as those with the UNIX operating system), the basic scheme that *Mathematica* uses to handle external programs is to run the programs and separate processes, and to communicate with them through pipes.

*Mathematica* is set up so that any commands which do input and output with files can also work with pipes.

| | |
|---|---|
| `<<`*file* | read in a file |
| `<<"!`*command*`"` | run an external command, and read in the output it produces |
| *expr* `>> "!`*command* | feed the textual form of *expr* to an external command |
| `ReadList["!`*command*`", Number]` | run an external command, and read in a list of the numbers it produces |

Some ways to communicate with external programs.

This feeds the expression `x^2 + y^2` as input to the external command `lpr`, which, on a typical UNIX system, sends output to a printer.

```
In[1]:= x^2 + y^2 >> "!lpr"
```

Putting a `!` at the beginning of a line causes the remainder of the line to be executed as an external command. `squares` is an external program which prints numbers and their squares.

```
In[2]:= !squares 4

1   1
2   4
3   9
4   16
```

This runs the external command `squares 4`, then reads pairs of numbers from the output it produces.

```
In[2]:= ReadList["!squares 4", {Number, Number}]
Out[2]= {{1, 1}, {2, 4}, {3, 9}, {4, 16}}
```